

```
1 #include "SortablePair.h"
2
3 template<typename K, typename V>
4 SortablePair<K, V>::SortablePair( const K& aFirst, const V& aSecond )    ↗
    noexcept
5     : fFirst(aFirst), fSecond(aSecond)
6 {
7
8 }
9
10 template<typename K, typename V>
11 const K& SortablePair<K, V>::first() const noexcept
12 {
13     return fFirst;
14 }
15
16 template<typename K, typename V>
17 const V& SortablePair<K, V>::second() const noexcept
18 {
19     return fSecond;
20 }
21
22 template<typename K, typename V>
23 bool SortablePair<K, V>::operator==( const SortablePair& aOther ) const    ↗
    noexcept
24 {
25     return fFirst == aOther.fFirst && fSecond == aOther.fSecond;
26 }
27
28
29 template<typename K, typename V>
30 bool SortablePair<K, V>::operator<( const SortablePair& aOther ) const    ↗
    noexcept
31 {
32     return fFirst > aOther.fFirst;
33 }
34
35
36
37 template class SortablePair<size_t, int>;
38 std::ostream& operator<<( std::ostream& aOStream, const    ↗
    SortablePair<size_t, int>& aPair )
39 {
40     aOStream << '(' << aPair.fFirst << ',' << aPair.fSecond << ')';
41     return aOStream;
42 }
43
```